

Phase 1: The Python Foundation

1. Core Syntax & Environment

- **The Interpreter:** How Python code is executed (Script mode vs. Interactive mode).
- **Comments:** Single-line (#) and Multi-line/Docstrings (""").
- **Indentation Rules:** The significance of whitespace in Python (PEP 8 standards).
- **Keywords & Identifiers:** Reserved words you cannot use as variable names.
- **Virtual Environments:** venv, pip, and requirements.txt management.

2. Variables & Memory Basics

- **Dynamic Typing:** Understanding that variables don't need declared types.
- **Naming Conventions:** Snake_case vs. camelCase.
- **Garbage Collection:** A brief look at how Python manages memory (Object references).

3. Data Types (Deep Dive)

- **Numeric:** int, float, complex.
- **Boolean:** True, False (and "Truthiness" of objects).
- **Strings:** * Indexing [0] and Slicing [start:stop:step].
 - Immutability (Why you can't change a character in a string).
 - Methods: .strip(), .split(), .join(), .replace(), .upper()/lower().
 - Formatting: f-strings (Modern), .format(), and % operator.
- **Industrial Coding Standards (PEP 8)**
 - Style Guides (PEP 8 / PEP 257): International rules for naming variables and formatting code.
 - First look at Auto-Formatters: Black and isort (one-click tools that make your code look professional).

4. Operators (The Math & Logic)

- **Arithmetic:** +, -, *, /, // (floor division), % (modulo), ** (exponent).
- **Assignment:** =, +=, -=, *=, etc.
- **Comparison:** ==, !=, >, <, >=, <=.
- **Logical:** and, or, not.
- **Identity & Membership:** is, is not, in, not in.

5. Advanced Data Structures (Collections)

- **Lists:** Nesting (Lists within lists), .append(), .extend(), .insert(), .pop(), .remove(), .sort().
- **Dictionaries:** Key-Value pairs, .keys(), .values(), .items(), .get(), handling nested dictionaries.
- **Tuples:** Packing and Unpacking, immutability use-cases.
- **Sets:** Uniqueness, set math (Union |, Intersection &, Difference -).
- **List/Dict Comprehensions:** Writing logic in a single line (Professional standard).

6. Control Flow & Logic

- **Conditionals:** if, elif, else and Nested ifs.
- **Ternary Operator:** x = a if condition else b.
- **Loops:**
 - for loops with range(), enumerate(), and zip().
 - while loops with infinite loop prevention.
 - break, continue, pass.
 - else clause in loops (A rare but professional topic).

7. Functions (The Modular Layer)

- **Parameters & Arguments:** Default values, Keyword arguments.

- **Return Statement:** Returning single vs. multiple values.
- **Lambda Functions:** Anonymous one-line functions.
- **Scope:** global vs. nonlocal keywords.
- **Recursion:** Functions calling themselves.

8. Error Handling & File I/O

- **Exception Hierarchy:** Catching specific errors vs. general errors.
- **Finally & Else:** The full try...except...else...finally block.
- **File Handling:** * open(), read(), write(), append().
 - The with statement (Context Managers)—the professional way to handle files.
 - Reading/Writing **CSV** and **JSON** specifically.

9. Advanced Functions (The "Expert" Level)

- **Arbitrary Arguments:** *args (non-keyword) and **kwargs (keyword arguments) for flexible functions.
- **Higher-Order Functions:** Passing functions as arguments to other functions.
- **Decorators:** The @decorator syntax (essential for Web Frameworks and Logging).
- **Closures:** Functions defined inside other functions that "remember" the outer scope.
- **Generators:** Using yield instead of return to handle large data without crashing memory.
- **Type Hinting:** Using def my_func(name: str) -> int: (The industry standard for clean code).

10. Advanced Loops & Iteration Tools

- **Iterators vs. Iterables:** Understanding __iter__ and __next__ under the hood.
- **The itertools Module:** Using chain, cycle, and product for complex logic.

- **Nested Loops:** Mastering multi-dimensional data (Matrices).
- **Loop Performance:** Why while True is used in game loops and AI listeners.

11. Object-Oriented Programming (The "Architecture" Layer)

- **Classes & Objects:** The blueprint vs. the actual instance.
- **The `__init__` Method:** Proper constructor usage and the self keyword.
- **Attributes:** Class attributes (shared) vs. Instance attributes (unique).
- **Methods:** * **Instance Methods:** Normal behavior.
 - **Class Methods (@classmethod):** Operating on the class level.
 - **Static Methods (@staticmethod):** Functions that live inside the class but don't need class data.
- **The Four Pillars of OOP:**
 1. **Inheritance:** Creating "Child" classes (e.g., Car inheriting from Vehicle).
 2. **Encapsulation:** Using private variables (name mangling `__variable`) to protect data.
 3. **Polymorphism:** Method Overriding (Different objects using the same function name differently).
 4. **Abstraction:** Using abc module to create "Abstract Base Classes" (Templates for other developers).
- **Dunder (Magic) Methods:** `__str__`, `__repr__`, `__len__`, `__add__` (Customizing how your objects behave with Python operators).

12. Modules & Packages (The "Organization" Layer)

- **Module Import System:** import, from ... import, and as aliases.
- **The `__name__ == "__main__"` Pattern:** Ensuring code only runs when the script is executed directly.
- **Package Structure:** Creating `__init__.py` files to turn folders into libraries.
- **Standard Library Exploration:** Deep look at os, sys, datetime, math, and random.

Phase 1: Performance-Based Tasks

Task 1: The Robust Numerical Processor

- **Topic:** Basic Logic & Error Handling.
- **Requirements:** Create a program that takes two numbers and an operator (\$+\$, \$-\$, \$*\$, \$/\$, \$**\$).
- **Deep Details:** * Wrap the input logic in a while True loop so the program doesn't close after one calculation.
 - Use try-except to catch ValueError (if the user types letters) and ZeroDivisionError.
 - The program must print a custom "Friendly Error" and ask for the numbers again without crashing.

Task 2: The "KPK District" Data Filter

- **Topic:** Collections & Comprehensions.
- **Requirements:** Create a list of dictionaries containing at least 10 KPK districts (Keys: name, population, is_hilly).
- **Deep Details:** * Use a **List Comprehension** to create a new list containing only the names of districts with a population > 1,000,000.
 - Use the .sort() method to arrange the final names alphabetically.
 - **Phase 2 Link:** This teaches data filtering which is essential for Data Science.

Task 3: Automated Directory "Janitor"

- **Topic:** File System & OS Module.

- **Requirements:** A script that cleans a messy folder by moving files based on their extensions.
- **Deep Details:** * Use `os.listdir()` to scan a folder.
 - Use `os.makedirs(exist_ok=True)` to create folders named Images, Docs, and Others.
 - Use `shutil.move()` to send `.jpg/.png` to Images and `.pdf/.docx` to Docs.

Task 4: The Persistence-Based Login System

- **Topic:** File I/O & Persistence.
- **Requirements:** Build a system that saves user credentials to a `.txt` or `.csv` file.
- **Deep Details:** * **Sign-up:** Append the username and password to the file, but only if the username doesn't already exist in the file.
 - **Login:** Read the file, loop through the lines, and check if the entered credentials match.
 - **Phase 2 Link:** This is the foundation for Database management.

Task 5: NLP Keyword Frequency Counter

- **Topic:** String Manipulation & Libraries.
- **Requirements:** Analyze a raw block of text to find the most important words.
- **Deep Details:** * Use `.lower()` and `.replace()` to remove punctuation (commas, periods, exclamation marks).
 - Use `.split()` to turn the text into a list of words.
 - Import Counter from collections to output the Top 5 most frequent words (excluding common words like "is", "the", "and").

Task 6: Modular Inventory Manager

- **Topic:** Functions & Scope.
- **Requirements:** Build an inventory system using only functions.
- **Deep Details:** * Define functions for `add_item(name, quantity)`, `update_stock(name, new_qty)`, and `view_all()`.
 - The inventory must be stored in a dictionary outside the functions.

- **Requirement:** Functions must use return statements to pass data back to the main loop.

Task 7: Live Currency Fetcher (Mock API)

- **Topic:** External Libraries (requests).
- **Requirements:** Fetch data from a web source.
- **Deep Details:** * Use pip install requests to set up the environment.
 - Use requests.get() to pull a exchange rate from a public URL or a "Mock API" (even a raw text file on GitHub).
 - Convert a hardcoded PKR amount to USD using the fetched rate.

Task 8: The Encapsulated "Digital Vault"

- **Topic:** OOP (Encapsulation).
- **Requirements:** Create a Vault class where the money is "hidden."
- **Deep Details:** * Use the double underscore __balance to make the variable private.
 - Create a @property (getter) to view the balance and a deposit method to increase it.
 - The withdraw method must return an error message if the user tries to take more than what is available.

Task 9: University Member Hierarchy

- **Topic:** OOP (Inheritance).
- **Requirements:** Model a school system with a parent and child classes.
- **Deep Details:** * **Parent Class:** Person (attributes: name, age).
 - **Child Classes:** Student (adds roll_no) and Teacher (adds subject).
 - Use super().__init__() to initialize the parent attributes within the child classes.

Task 10: The CLI "Task Master" (Final Phase 1 Project)

- **Topic:** Full Integration.
- **Requirements:** A text-based Menu app to manage daily tasks.

- **Deep Details: * The Menu:** Print a numbered list: 1. Add Task, 2. View Tasks, 3. Save & Exit.
 - **Functionality:** Use a class to manage the tasks and a .json file to save the data so it isn't lost when the program closes.
 - **Logic:** Combine Loops, Classes, and File I/O into one single script.